Python Quiz 4 - MSD Practice

Instructions:

For each example:

- Draw the memory-state diagrams and write the exact output.
- Carefully track which variables refer to the same list and when new lists are created.

What You Should Already Know

Rule	Meaning	Example
1	Element assignment mutates an existing list.	`data[0] = 5`
2	`append()` mutates an existing list.	`data.append(5)`
3	Slicing creates a new list.	`new_data = data[1:]` `# also [::2] and [::-1]`
4	Concatenation creates a new list.	`new_data = data + [5]`
5	Assignment alone does not create a new list. Both names refer to the same list.	`alias = data`
6	Reassigning a parameter does not reassign the caller's variable.	`data = data[1:]`
7	Returning a list returns a reference to that list.	`return data`
8	Python evaluates the right-hand side before performing the assignment.	`data[0] = guide[1] + 1`

Example 1 - Mutation followed by concatenation

Draw the memory-state diagrams for the program and write its exact output.

```
def update(items):
    position = 1

    while position < len(items):
        items[position] *= 2
        position += 2

    new_items = items + [items[0] - 1]
    return new_items


values = [4, 3, 6, 5]
result = update(values)

print(values)
print(result)
```

Expected output:

Memory-state diagram space:

Example 2 - Creating and modifying a slice

Draw the memory-state diagrams for the program and write its exact output.

```
def adjust(data):
    selected = data[1:]
    position = 0

    while position < len(selected):
        selected[position] += position + 1
        position += 1

    return selected


values = [3, 5, 7, 9]
result = adjust(values)

print(values)
print(result)
```

Expected output:

Memory-state diagram space:

Example 3 - Preserving an alias before reassignment

Draw the memory-state diagrams for the program and write its exact output.

```
def revise(numbers):
    original = numbers
    numbers = numbers[1:]

    position = 0
    while position < len(numbers):
        numbers[position] -= 1
        position += 2

    original[1] = numbers[-1]
    return numbers


values = [4, 7, 10, 13]
result = revise(values)

print(values)
print(result)
```

Expected output:

Memory-state diagram space:

Example 4 - Returning the original list

Draw the memory-state diagrams for the program and write its exact output.

```
def replace(data):
    guide = data[1:]
    position = 0

    while position < len(guide):
        data[position] = guide[position] + 1
        position += 1

    return data


values = [2, 5, 8, 11]
result = replace(values)

print(values)
result[-1] = result[0]
print(values)
```

Expected output:

Memory-state diagram space:

Example 5 - Coordinating changes to two separate lists

Draw the memory-state diagrams for the program and write its exact output.

```
def combine(data):
    part = data[::2]
    position = 0

    while position < len(part):
        part[position] += data[position + 1]
        position += 1

    data[0] = part[-1]
    part.append(data[-1])
    return part


values = [2, 4, 6, 8]
result = combine(values)

print(values)
print(result)
```

Expected output:

Memory-state diagram space: